

The ESP32-S3 & MicroPython

A Story-Driven Guide for the BikeWheel POV Project

This guide explains how the XIAO ESP32-S3 microcontroller works, why certain pins were chosen for SPI, hall sensor, and WiFi, and how MicroPython sits on top of it all to bring your code to life. No prior electronics degree required.

Chapter 1: What Is the ESP32-S3?

Imagine a very small, very capable computer that fits on your fingernail. That's the ESP32-S3. It's a microcontroller — a chip that contains a processor, memory, and a bunch of input/output pins, all in one tiny package. It was designed by a Chinese company called Espressif and is used in everything from smart home devices to industrial sensors to, in your case, a spinning LED bike wheel.

What's Inside the Chip?

The ESP32-S3 has several key components all baked into a single chip:

Component	What it does	Analogy
Dual-core CPU (Xtensa LX7)	Runs your code at up to 240MHz	The brain — thinks fast
512KB SRAM	Working memory — holds variables while code runs	Your desk — stuff you're currently working on
384KB ROM	Stores bootloader — the code that starts everything up	The startup manual you can't change
8MB Flash (on XIAO)	Stores your Python files permanently	Your hard drive
WiFi radio	2.4GHz 802.11 b/g/n — runs your BikeWheel hotspot	Built-in phone radio
Bluetooth 5.0	Short range wireless (not used in your project)	Optional walkie-talkie
45 GPIO pins	Connect to LEDs, sensors, batteries — the real world	Hands reaching out to the world

What Makes the S3 Special?

The "S3" variant was released in 2021 and brought some important upgrades over earlier ESP32 chips. Most importantly for your project, it has a much faster CPU (240MHz dual core), better SPI hardware, and native USB support — which is why you can plug the XIAO directly into your laptop and Thonny can talk to it without needing a separate USB-to-serial adapter chip.

The XIAO Form Factor

Seeed Studio's XIAO ESP32-S3 is a particular board design around the ESP32-S3 chip. At just 21x17mm it is one of the smallest ESP32-S3 boards available. This is why it fits in your hub enclosure where a standard development board never would. It breaks out 11 GPIO pins, has a USB-C port, a built-in antenna, a charge LED, and a reset button — everything you need, nothing you don't.

Chapter 2: GPIO Pins — The Hands of the Chip

GPIO stands for General Purpose Input/Output. These are the physical pins on the chip that connect to the outside world — your LEDs, your hall sensor, your power rails. Think of them as the chip's hands. Some hands can only receive (input), some can only give (output), and most can do both.

Input vs Output

When a pin is configured as an INPUT, the chip reads a voltage on that pin and decides if it's HIGH (3.3V = logical 1) or LOW (0V = logical 0). Your hall sensor output connects to an input pin — the chip reads whether the magnet is present or not.

When a pin is configured as an OUTPUT, the chip drives the pin to 3.3V or 0V. This is how it sends data and clock signals to the LED strip — by rapidly toggling output pins high and low in precise patterns.

Pull-up and Pull-down Resistors

Here's a subtle but important concept. When nothing is connected to an input pin, what does it read? The answer is "garbage" — it floats around randomly. To prevent this, you can activate an internal resistor that gently pulls the pin to a known state.

A PULL_UP resistor connects the pin to 3.3V through a large resistor (~45k ohms). So the pin reads HIGH (1) by default, and only goes LOW (0) when something actively pulls it to ground. This is exactly why your hall sensor circuit works: the pin sits at 1 normally, and drops to 0 when the magnet passes. In MicroPython:

```
pin = Pin(4, Pin.IN, Pin.PULL_UP)
```

The XIAO ESP32-S3 Pinout

The XIAO has 11 exposed GPIO pins. Here is every pin and what it does in your project:

XIAO Label	GPIO #	Your Project	Why This Pin
5V	—	Power to LEDs + MT3608 output	Only 5V power pin on board
GND	—	Common ground for all components	Ground reference
3V3	—	Hall sensor VCC	3.3V — matches sensor rating
D0	GPIO1	Available	—
D1	GPIO2	Available (status LED future)	General purpose output
D2	GPIO3	Available	—
D3	GPIO4	Hall sensor OUT	See Chapter 3 — interrupt capable
D4	GPIO5	Available (I2C SDA)	—
D5	GPIO6	Available (I2C SCL)	—

D8	GPIO7	SK9822 Clock (CI / Yellow wire)	Hardware SPI SCK — see Chapter 4
D9	GPIO8	Available	—
D10	GPIO9	SK9822 Data (DI / Green wire)	Hardware SPI MOSI — see Chapter 4

Purple rows = SPI pins (hardware accelerated). Yellow row = interrupt-capable pin used for hall sensor.

Chapter 3: Interrupts — How the Hall Sensor Works

The Problem With Polling

Imagine you're cooking dinner and waiting for a friend to knock on the door. You could check the door every 5 seconds — walk to the door, check, walk back, cook, walk to the door, check, walk back... This is called polling and it wastes most of your time walking to the door for nothing.

Your POV loop has the same problem. It's busy displaying 60 columns of LEDs with microsecond precision. It cannot stop every few milliseconds to check if the hall sensor fired — that would ruin the timing and blur the image.

The Solution: Hardware Interrupts

An interrupt is the solution. Instead of your code checking the sensor, you tell the hardware: "the moment that pin goes from HIGH to LOW, stop everything and run this function immediately." Then go back to what you were doing.

It's like installing a doorbell instead of checking the door every 5 seconds. You cook normally. The doorbell rings. You answer it. You go back to cooking. The chip's hardware handles this at a level below your Python code — faster and more reliable than anything you could write in software.

Why GPIO4 (D3)?

On the ESP32-S3, almost all GPIO pins are capable of generating interrupts. GPIO4 was chosen simply because it's available, clearly labeled as D3 on the XIAO, and not needed for SPI or any other dedicated function. There's nothing magic about GPIO4 specifically — any free pin would work.

The Falling Edge

We configure the interrupt to trigger on a FALLING EDGE — the moment the signal transitions from HIGH (1) to LOW (0). That's the exact instant the magnet passes the sensor. Not the rising edge (LOW to HIGH, when the magnet moves away), not a level trigger (fires continuously while LOW) — just that one precise moment of transition. In MicroPython:

```
pin.irq(trigger=Pin.IRQ_FALLING, handler=self._on_trigger)
```

The 5ms Debounce

Physical magnets and sensors produce noisy signals. When the magnet passes, the signal doesn't cleanly go LOW once — it can bounce up and down several times in a few microseconds before settling. Without protection, this looks like multiple rapid triggers and your rotation count would be wrong.

The debounce check ($\text{diff} > 5000$ microseconds) ignores any trigger that happens within 5ms of the previous one. At maximum wheel speed — say 200 RPM — one full rotation takes 300,000 microseconds. So any trigger within 5ms of the last one is definitely noise, not a real rotation.

Chapter 4: SPI — How Data Gets to the LEDs

What is SPI?

SPI stands for Serial Peripheral Interface. It's a communication protocol — a set of rules for how two devices talk to each other. SPI uses two data wires and is extremely fast, which is why the SK9822 LEDs use it.

Think of SPI like a very fast conveyor belt with a timing light. The clock wire flashes a light at regular intervals. Every time the light flashes, the data wire carries one bit of information. The LED strip reads each bit in sync with each clock flash. This synchronization is what makes SPI so reliable — both sides always agree on exactly when to read the data.

The Two SPI Wires

Wire	SPI Name	XIAO Pin	SK9822 Label	Color	What it does
Clock	SCK	D8 / GPIO7	CI (Clock In)	Yellow	Beats like a metronome — one beat per bit
Data	MOSI	D10 / GPIO9	DI (Data In)	Green	Carries the actual color data — one bit per beat

Why Hardware SPI? (GPIO7 and GPIO9)

The ESP32-S3 has dedicated SPI hardware built into the chip — dedicated circuits that handle the clock and data signals automatically at up to 80MHz. GPIO7 and GPIO9 are directly wired to these hardware SPI circuits.

You could theoretically use any GPIO pins for SPI by manually toggling them in Python code — this is called "bit-banging." But there's a critical problem: MicroPython is an interpreted language running on top of an operating system. It cannot toggle pins fast enough or consistently enough for reliable high-speed SPI.

By using GPIO7 and GPIO9 (the hardware SPI pins), you hand the job to dedicated silicon circuits that run independently of your Python code at 8 million bits per second. Your Python just says "send this data" and the hardware does the rest instantly. This is why your full strip of 72 LEDs updates in about 288 microseconds — fast enough for precise POV timing.

Key insight: GPIO7 and GPIO9 were chosen not because of the numbers, but because they are physically wired inside the chip to the hardware SPI peripheral. Using any other pins for SPI would require slow, unreliable software bit-banging.

SPI Mode 0 — Why polarity=0, phase=0?

SPI has four modes (0-3) that define when data is valid relative to the clock. The SK9822 datasheet specifies Mode 0: clock idles LOW (polarity=0) and data is read on the rising edge (phase=0). These

are not arbitrary choices — they must match what the LED chip expects or the data gets corrupted.

The SK9822 Frame Format

The SK9822 has a very specific format for how color data must be sent. For every LED update, the XIAO sends a stream of bytes in this exact order:

Section	Bytes	Content	Purpose
Start frame	4 bytes	00 00 00 00	Signals new data coming — "attention!"
LED 1 frame	4 bytes	FF bb gg rr	Brightness + Blue + Green + Red for LED 1
LED 2 frame	4 bytes	FF bb gg rr	Data for LED 2
... (repeat)	4 bytes each	...	One frame per LED
End frame	4+ bytes	FF FF FF...	Clocks data through to last LED in chain

Note: SK9822 sends color in Blue-Green-Red order, NOT Red-Green-Blue. This is why `apa102.py` packs bytes as `[brightness, b, g, r]` not `[brightness, r, g, b]`.

Chapter 5: MicroPython — The Language Layer

What is MicroPython?

MicroPython is a lean, efficient implementation of Python 3 designed to run directly on microcontrollers. Regular Python runs on your laptop with gigabytes of RAM and a full operating system underneath it. MicroPython runs on a chip with 512KB of RAM and no operating system — just bare metal.

Think of it like this: regular Python is a luxury car with all the amenities. MicroPython is a stripped-down racing car — lighter, faster to start, runs on less fuel, but still recognizably a car that you drive the same way.

The Software Stack

When you power on the XIAO, here is exactly what happens in order:

Layer	What it is	What it does
1. Hardware	ESP32-S3 silicon chip	Electricity flows, transistors switch
2. Bootloader	Stored in ROM (read-only)	Initialises hardware, finds MicroPython in flash
3. MicroPython VM	Stored in your 8MB flash	Interprets Python bytecode, manages memory
4. boot.py	Your file in flash	Runs first — WiFi setup, startup config
5. main.py	Your file in flash	Runs second — your actual program
6. Your modules	apa102.py, hall_sync.py etc	Imported by main.py as needed

The machine Module — Talking to Hardware

MicroPython's most important feature for hardware projects is the machine module. This is a special module that provides direct access to the chip's hardware peripherals from Python. Without it, you'd need to write C code to access the GPIO pins and SPI bus.

Class	What it controls	Used in your project
machine.Pin	Individual GPIO pins	Hall sensor input with pull-up and interrupt
machine.SPI	Hardware SPI bus	Sending color data to SK9822 LEDs at 8MHz
machine.I2C	I2C communication bus	Not used yet — available for sensors
machine.PWM	Pulse width modulation	Not used — could control motor speed etc
machine.Timer	Hardware timers	Not used — interrupts handle timing instead
machine.ADC	Analog to digital converter	Could read battery voltage in future

Interpreted vs Compiled — Why MicroPython is Slower

Regular C code that runs on microcontrollers is compiled — translated directly into machine instructions before it runs. Every line of C becomes raw binary instructions that the CPU executes directly.

MicroPython is interpreted — your Python code is read line by line at runtime and translated on the fly. This is slower, but not by as much as you'd think for most tasks. Where it matters is timing-critical operations like bit-banging SPI.

This is exactly why your project uses hardware SPI (`machine.SPI`) instead of manually toggling pins in Python. The hardware SPI runs in dedicated silicon independent of the Python interpreter. Your Python just triggers it and the hardware does the rest at full speed.

`ticks_us()` — Microsecond Timing

The POV column timing requires microsecond precision — each column must display for exactly the right number of microseconds or the image blurs. MicroPython provides `time.ticks_us()` which returns the current time in microseconds.

One subtlety: the tick counter is not infinite. It wraps around after a while (about 17 minutes on ESP32). This is why the code uses `time.ticks_diff()` to calculate time differences instead of simple subtraction. `ticks_diff()` handles the wrap-around correctly so timing never breaks after 17 minutes of running.

Chapter 6: WiFi — The Built-in Radio

One of the most remarkable things about the ESP32-S3 is that a full 2.4GHz WiFi radio is built directly into the chip at no extra cost or size. The same silicon that runs your Python code also broadcasts a WiFi hotspot that your Android app connects to.

Access Point vs Station Mode

The ESP32 WiFi can operate in two modes. In Station mode (STA_IF) it behaves like your phone — connecting to an existing WiFi network like your home router. In Access Point mode (AP_IF) it creates its own network that other devices connect to. This is what your project uses, because on the Burning Man playa there is no WiFi network to connect to. The wheel IS the WiFi network.

The Web Server on Port 80

Your Android app sends HTTP requests to `http://192.168.4.1/pov` or `/safety` or `/brightness/25` etc. The XIAO's web server listens on port 80 (the standard HTTP port) and responds to these requests. It's a stripped-down version of what a web server like Apache or Nginx does — but implemented in about 20 lines of Python instead of millions.

The key design choice is non-blocking mode. The socket is set to `setblocking(False)` so the web server check never freezes the main loop. The POV timing loop can keep running while the WiFi waits for requests. Without this, checking for web requests would pause the LEDs and ruin the POV effect.

Chapter 7: Power — Keeping Everything Running

The ESP32-S3 operates at 3.3V logic level. This means all its GPIO pins output 3.3V for HIGH and 0V for LOW. However, your LED strip needs 5V to operate correctly. This mismatch is why the MT3608 boost converter exists.

The Two Circuits

Circuit	Path	Purpose
Riding (power)	LiPo 3.7V → MT3608 → 5.07V → XIAO 5V pin + LED strip	Powers everything while riding
Charging	USB-C wall charger → TP4056 → LiPo battery	Charges battery at camp overnight

Critical rule: the TP4056 charger is NEVER in the riding power circuit. It connects only to the battery for charging. Mixing them would damage the TP4056.

Voltage Logic Levels and the LED Strip

The XIAO outputs 3.3V logic on its GPIO pins. The SK9822 technically expects 5V logic signals on DI and CI. In practice, the SK9822 accepts 3.3V signals as valid HIGH — which is why the project works without a level shifter. If you ever experience unreliable first-LED behavior, a 330 ohm resistor on the DI line or a proper logic level shifter would fix it.

Current Budget

At full brightness, each SK9822 LED draws about 60mA (20mA per color channel). With 72 LEDs that's theoretically 4.3A. In practice, your POV display never lights all LEDs at full white simultaneously — typical current draw is closer to 500mA-1A for colorful animation frames. The LiPo 2000mAh battery gives you roughly 2-4 hours of riding on a single charge.

Quick Reference Card

Pin Summary

What	XIAO Label	GPIO	Why
LED Clock (CI)	D8	GPIO7	Hardware SPI SCK pin
LED Data (DI)	D10	GPIO9	Hardware SPI MOSI pin
Hall Sensor OUT	D3	GPIO4	Interrupt capable, free pin
Hall Sensor VCC	3V3	—	3.3V matches A3144 rating
LED Strip 5V	5V	—	Only 5V output on board
All GND	GND	—	Common ground

Key Concepts Cheat Sheet

GPIO — General Purpose Input/Output — the physical pins on the chip that connect to the real world

SPI — Serial Peripheral Interface — two-wire protocol (clock + data) used to drive the SK9822 LEDs

Hardware SPI — SPI handled by dedicated silicon circuits at up to 80MHz — much faster than software

Interrupt — Hardware mechanism that fires a function instantly when a pin changes — how hall sensor works

Pull-up resistor — Holds a pin HIGH by default — hall sensor pulls it LOW when magnet passes

Falling edge — The moment a pin transitions HIGH to LOW — when the interrupt fires

Debounce — Ignoring triggers within 5ms of each other — filters out electrical noise

AP mode — Access Point — XIAO creates its own WiFi network for your phone to connect to

Non-blocking socket — Web server that checks for requests without freezing — critical for POV timing

ticks_us() — MicroPython microsecond timer — used for precise POV column timing

MicroPython VM — The interpreter that runs your .py files on the bare metal chip

boot.py / main.py — Auto-run files — boot.py first, then main.py, every time the chip powers on